

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и
информатики»

Кафедра телекоммуникационных систем и вычислительных средств
(ТС и ВС)

Отчет по производственной практике
Занятие №3

по теме:

ПРИНЦИПЫ РАБОТЫ БИБЛИОТЕКИ SOAPY SDR И РАБОТЫ С ADALM
PLUTO. РАБОТА С БИБЛИОТЕКАМИ SOAPY SDR, LIBIO
ФОРМИРОВАНИЕ И ПЕРЕДАЧА С SDR СИГНАЛОВ ПРОИЗВОЛЬНОЙ
ФОРМЫ

Студент:
Группа ИА-331

Я.А Гмыря

Предподаватели:
Лектор
Семинарист
Семинарист

Ахнашев А.В
Ахнашев А.В
Попович И.А

Новосибирск 2025 г.

СОДЕРЖАНИЕ

1	ВВЕДЕНИЕ	3
1.1	Введение	3
1.2	Цель	3
2	ЛЕКЦИЯ	4
2.1	Структура семплов в Pluto SDR	4
2.2	Структура буфера семплов в Pluto SDR	5
2.3	Запись RX семплов в буфер с Pluto SDR	5
2.4	Создание своих семплов и их отправка в Pluto SDR	6
3	ПРАКТИКА	7
3.1	Формирование семплов	7
3.1.1	Перевод строки в бинарный вид	7
3.1.2	Способ кодирования	8
3.1.3	Заполнение буфера семплами	8
3.2	Результат работы	10
3.2.1	Семплы до отправки	10
3.2.2	Требование к длине сообщения	10
3.3	Семплы на приеме	10
4	ВЫВОД	12

ВВЕДЕНИЕ

1.1 Введение

На прошлом занятии мы отправляли, а потом принимали семплы и визуализировали их, работая с SDR с помощью C/C++ и библиотеки SoapySDR. На этом занятии сформируем семплы произвольной формы, отправим их и визуализируем.

1.2 Цель

Лучше освоить работу с SDR с помощью C/C++ и библиотеки SoapySDR, сформировать собственные семплы произвольной формы, отправить их и визуализировать после приема.

ЛЕКЦИЯ

2.1 Структура семплов в Pluto SDR

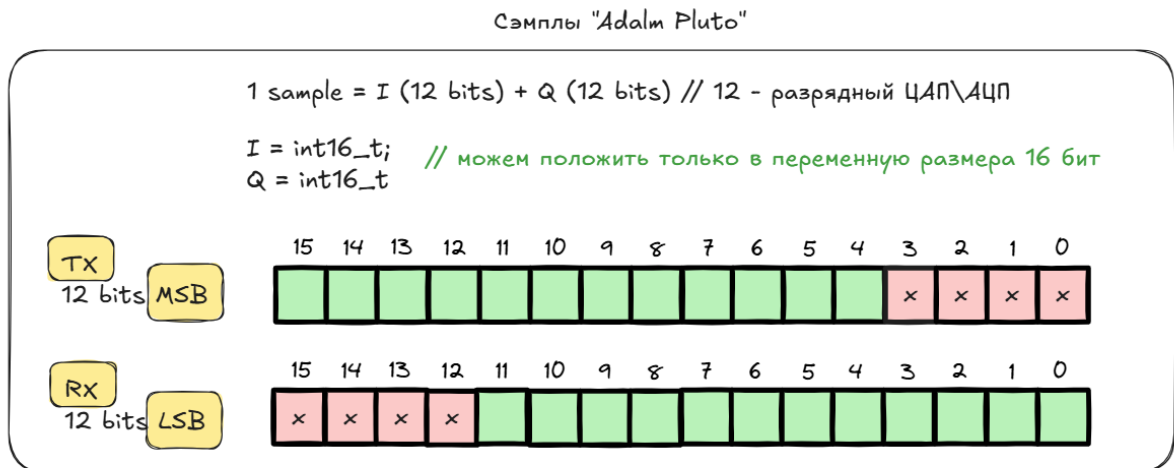


Рисунок 1 — Структура семплов

Pluto SDR имеет 12-ти битный АЦП, это значит, что для I и Q максимальное значение составляет $2^{12} = 4096$, но один бит займет знак, поэтому I и Q будут принимать значения из отрезка $[-2048; 2047]$. Для хранения I или Q в C++ используется тип данных `int16_t` (если брать меньше, то семпл не поместится). Таким образом один семпл занимает 4 байта памяти. Также есть вариант хранить семплы в одной переменной `int32_t`, но в таком случае для получения доступа к I или Q придется использовать битовые сдвиги и накладывать маски.

Стоит отметить, что Pluto SDR странно работает с TX семплами (которые хотим передавать) и интерпретировать как семплы только первые 12 бит переменной (big-indian), поэтому необходимо сдвигать значение I и Q на 4 бита влево («4») при работе с tx буффером, чтобы Pluto SDR корректно их интерпретировать.

2.2 Структура буфера семплов в Pluto SDR

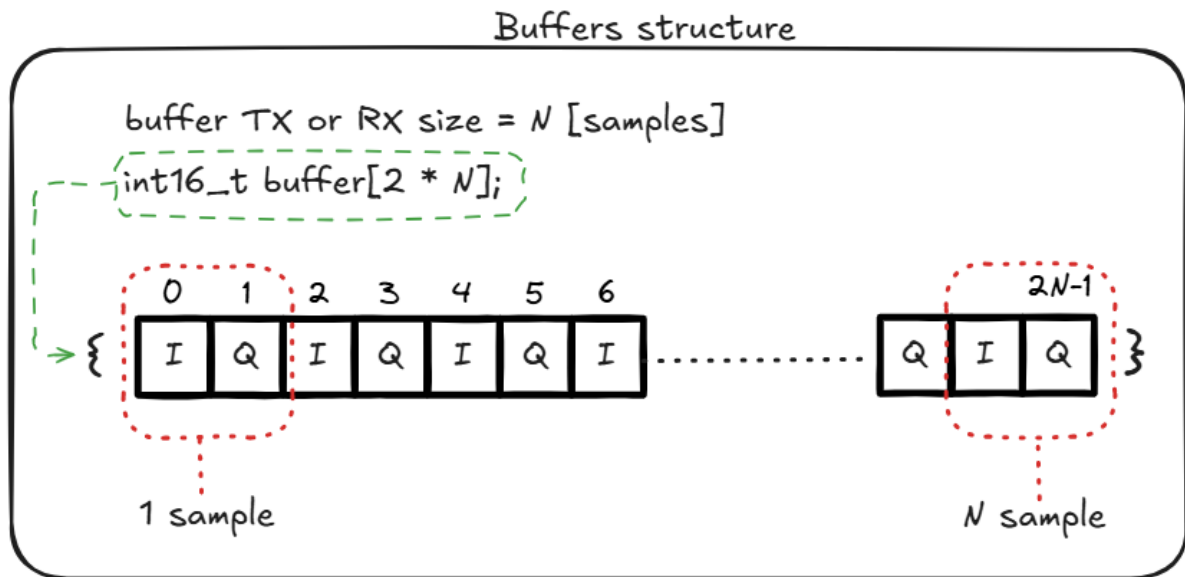


Рисунок 2 — Структура буфера

Семпл состоит из компонент I и Q, которые хранятся последовательно друг за другом, т.е в буфер будет иметь последовательность вида $I_0, Q_0, I_1, Q_1, \dots, I_n, Q_n$, поэтому если мы хотим принять/передать N семплов, то буфер должен быть размером $2N$, т.к семпл состоит из двух чисел.

2.3 Запись RX семплов в буфер с Pluto SDR

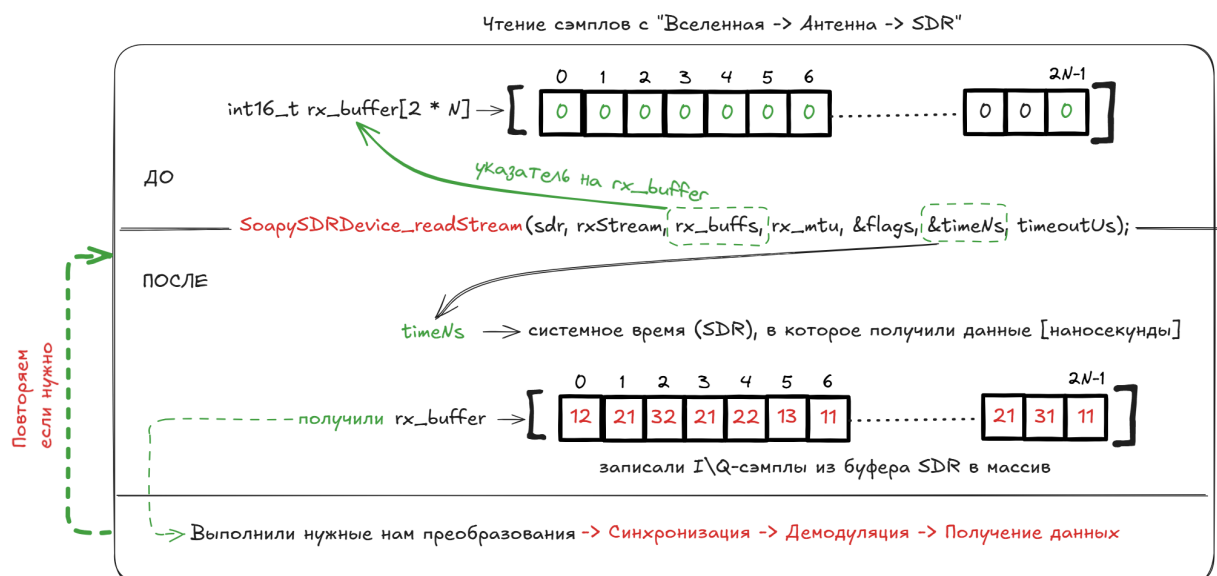


Рисунок 3 — Запись RX семплов

Для записи RX семплов в буффер используется функция `SoapySDRDevice_readStream()`, в которую необходимо передать указатель на буффер, в который хотим писать, и кол-во семплов, которые мы хотим записать, и еще некоторые дополнительные параметры. После выполнение функции в наш буффер будут записаны семплы, принятые SDR.

2.4 Создание своих семплов и их отправка в Pluto SDR

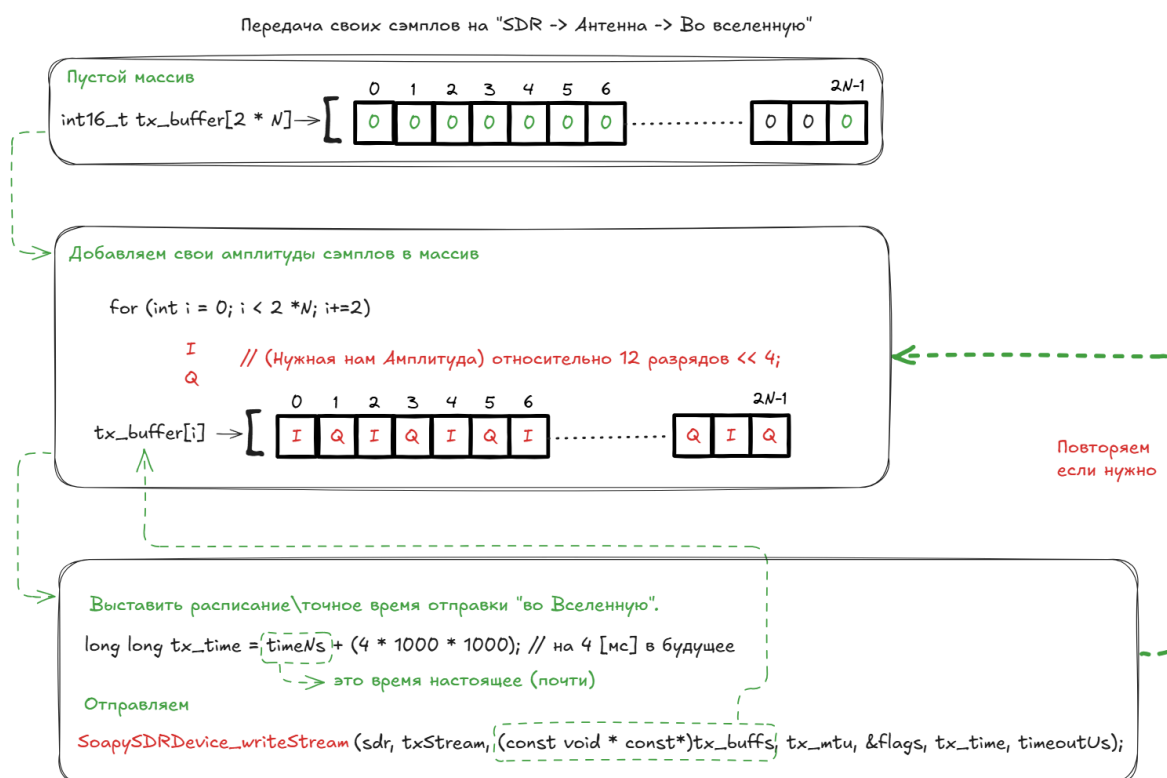


Рисунок 4 — Создание и отправка семплов

Для отправки семплов сначала необходимо заполнить tx буффер самими семплами. Формировать сами I и Q можно разными способами, но самое главное разместить их в правильном порядке. Для самой отправки используется функция `SoapySDRDevice_writeStream()`, в которую необходимо передать указатель на буффер с семплами и время, через которое семплы будут отправлены, а также дополнительные параметры. После вызова функции через 4нс наши семплы отправятся с Pluto SDR.

ПРАКТИКА

3.1 Формирование семплов

3.1.1 Перевод строки в бинарный вид

Я буду формировать простой прямоугольный сигнал, который будет передавать небольшую строку. Чтобы передавать текст, нужно сначала перевести символы (char) в биты. Для этого я написал функцию:

```
int8_t* stob(char* str, int* out_bits_count) {
    // get string len
    int len = strlen(str);

    // 1 char = 8 bits
    *out_bits_count = len * sizeof(char);

    // array for bits
    int8_t* bits = (int8_t*)malloc(*out_bits_count *
        sizeof(int8_t));

    // check pointer
    if (bits == NULL){
        return NULL;
    }

    char c;

    // iterate on string
    for (int i = 0; i < len; i++) {
        // get char
        c = str[i];
        // iterate on bits array
        for (int j = 0; j < 8; j++) {
            // convert char to bits
            bits[i * 8 + j] = (c >> (7 - j)) & 1;
        }
    }

    return bits;
}
```

```
}
```

Функция принимает саму строку и указатель на переменную, в которой вернет количество бит, чтобы знать размер битовой последовательности после завершения функции, и возвращает битовую последовательность.

Пример работы функции:

Переведем в бинарный вид строку "Hello World":

```
int bits_count;  
uint8_t* bits = stob("Hello World", &bits_count);
```

На выходе получим такую последовательность:

```
0100100001100101011011000110110001101110010000001010111011011110111  
00100110110001100100
```

Рисунок 5 — "Hello World" в бинарном виде

3.1.2 Способ кодирования

Чтобы передать нули и единицы я буду использовать самый простой способ кодирования: 0 - ноль, 1 - максимальный I и минимальный Q.

3.1.3 Заполнение буфера семплами

Для заполнения tx буфера я написал отдельную функцию, которая принимает битовую последовательность (сообщение), длину битовой последовательности и размер буфера, а возвращает массив семплов.

```
int16_t* bits_to_samples(uint8_t* bits, int bits_count, int  
tx_mtu){  
  
    // allocate memory  
    int16_t* tx_buff = (int16_t*)malloc(sizeof(int16_t) * tx_mtu  
        * 2);  
  
    // iterate on bitts  
    for (int i = 0; i < bits_count; i += 1)  
    {  
        // fill tx_buff with samples
```



```

for(int j = i*TAU_ON_BITS; j < i*TAU_ON_EL + 20 && j <
tx_mtu*2; j+=2){
    if(bits[i]){
        tx_buff[j] = 2047 << 4;    // I
        tx_buff[j+1] = -2047 << 4; // Q
    } else{
        tx_buff[j] = 0;            //I
        tx_buff[j+1] = 0;          //Q
    }
}
}

return tx_buff;
}

```

Самое важное здесь - учитывать продолжительность импульса, т.е то кол-во семплов, которое будет приходиться на один бит информации. Я выбрал 10 семплов на бит (TAU), а это значит, что на 1 бит информации будет приходиться 20 элементов массива (TAU_ON_EL). Чем больше длительность сигнала, тем выше шанс верно декодировать его на приемной стороне, но при этом падает скорость передачи данных. Работать заполнение будет так: берем первые 20 элементов массива и заполняем его идентичными значениями (в соответствии со состоянием бита), потом берем следующие 20 и т.д.

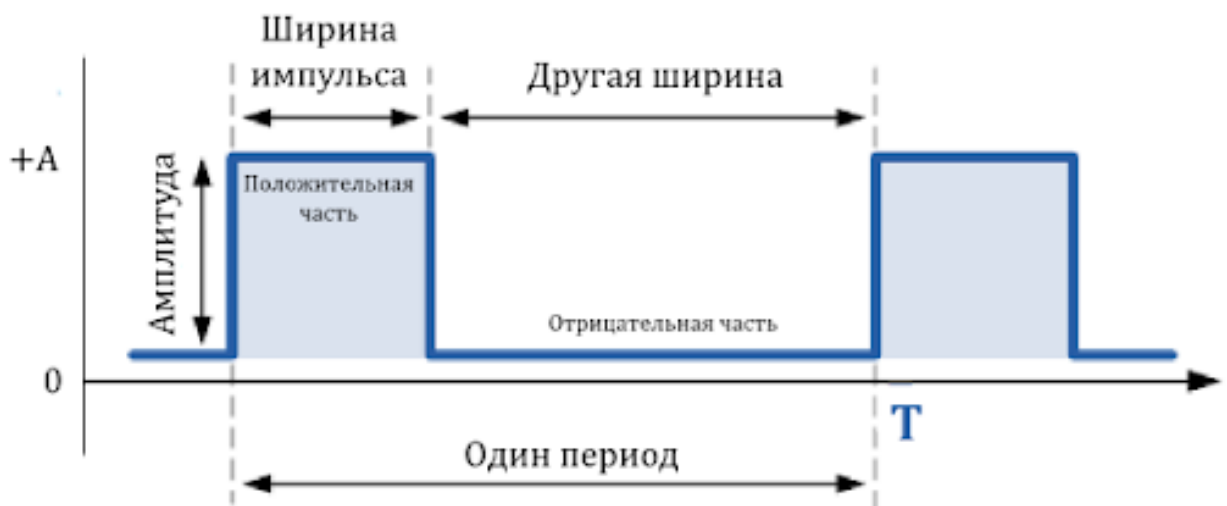


Рисунок 6 — Пример прямоугольного сигнала с обозначениями

3.2 Результат работы

3.2.1 Семплы до отправки

С помощью Python визуализируем семплы

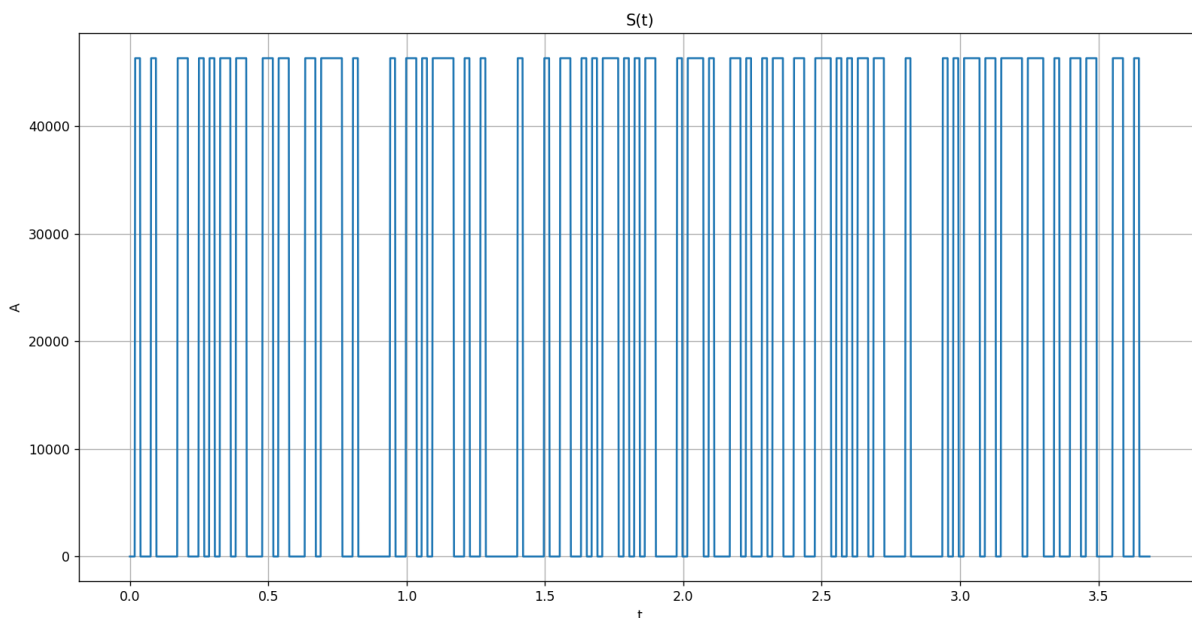


Рисунок 7 — Пример прямоугольного сигнала

Этот прямоугольный сигнал содержит наше сообщение.

3.2.2 Требование к длине сообщения

При работе с Pluto SDR рекомендуется использовать для отправки/приема 1920 семплов или число семплов кратное 1920. Если на 1 бит приходится 10 семплов, то сообщение должно быть длиной 192 бита, а т.к. я пытаюсь передать символы размером 8 бит, то необходима строка длиной 24 символа или строка с длиной кратной 24.

3.3 Семплы на приеме

На приеме получили следующий сигнал:

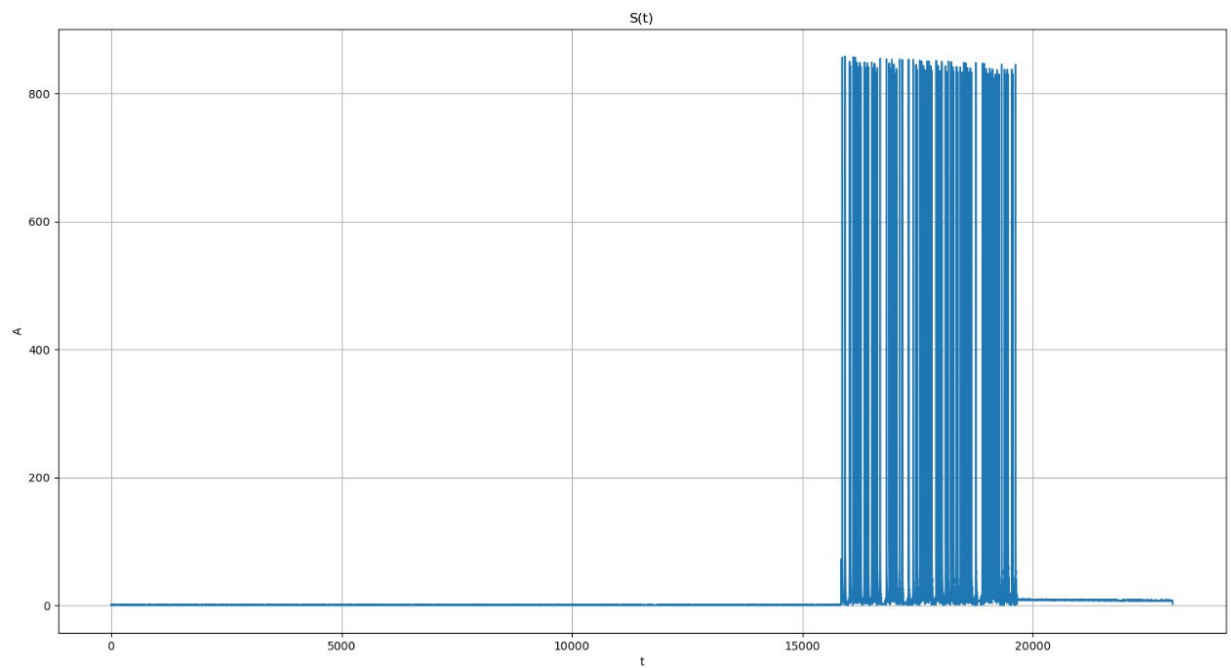


Рисунок 8 — Сигнал на приеме

Где то среди этого сигнала наше сообщение, но чтобы правильно его декодировать, необходимо реализовать логику приемной стороны и выполнить символьную синхронизацию. Эти темы будет рассматриваться в дальнейшем.

ВЫВОД

В ходе проделанной работы я улучшил свои навыки работы с SDR с помощью C/C++ и библиотеки SoapySDR. Научился формировать семплы и отправлять/принимать их с SDR.